Date __/__/____

Lec - 103

(Bg color changer) Project

root → container → h1
→ input
→ button
→ div (tasks) → h2
→ div (inner content) → p
→ button (delete)

when ever there is state change of a variable or anything.

use use state.

↳ in colorful.isl component

let [color, setColor] = useState("white")

where ever there is frequent change in state of a variable, Then use use state.

Now consider a ~~Eg~~ Senario →

<button onClick = {() => setColor("red")}

↓

when you click on red,

The SetColor function will do re-rendering of function

Page No. ____

Date , See commit (without use Effect hook)

re-rendering means calling function one more time, function in which the called (setColor) function is inside.

here: ~~main~~ (~~colorful~~) (2 function will be called again
main
(see github)

Now you can see -

same bth

if you click a button two times then it will render (see console)

but not more than 2 times.

So, here let [color, setColor] = useState("white")

it is Primitive data type (string)

So, ~~it~~ useState re-render it one more time even if the value doesn't change.

but if state variable will be of non-Primitive → (object, array)

Then it will render only 1 time. b/c of heap memory.

That's why Destructuring is used in Myntra project

Date

reasoning behind it → They have given it as bailout term.

That Primitive data type are allowed to upto 2 re-rendering without any state-change. (content change)

↳ non-Primitive (array, object) are not allowed if content (state) is not changed.

You can see in in commit (without use Effect)

That
[document.body.style.backgroundColor = color;

This is open. (and down it manipulating multiple times)

blc of re-rendering.

Now we will use →

Use effect hook

3rd commit (Use of use Effect hook)

We are using use Effect blc of re-rendering of code without use Effect.

Page No.

(4) Date __/__/____

UseEffect hook

React hook that lets you perform side effects in a functional component.

A side effect is anything your component does outside of rendering the UI.

Ex→ changing document body styles, fetching data from API, etc.

Syntax→

```
import {useEffect} from "react";

useEffect(() => {
        // side effect code
    }, []);
```

The first argument is a function that contains the side effect.

The second argument is the dependency array→ which tells React when to run the effect.

example→

```
useEffect(() => {
        log("Run");
    }, []);
```

Date 4/__/__

Dependency Array.

(1) Empty array []

```
useEffect(() => {
    // log
}, []);
```

Runs only once after the component mounts.

(2) one (or more) dependency

```
useEffect(() => {
    // log
}, [count]);
```

Runs →
- on first render
- whenever count changes.

Or in multiple dependencies → runs whenever either count or name changes.

(3) No dependency array.

(Runs after every Render.)

```
useEffect(() => {
    // log
});
```

Page No.

6 Date __/__/____

Here →

```
useEffect(() => {
    document.body.style.bgc = color;
}, [color]);
```

for rendering it only when color changes

? Advantage of using useEffect here →

it will not render 2 times when clicking to same button.

so, statement will not render in repetition.

Note → useEffect is executed in last in its file. always - (in whole component)

as you can see in (console) after rendering.

① first
② second
③ useEffect executed

S.T.M is written above

output ① ③ ② → order in is

Date ___/___/___

Note

→ Use useEffect for side effects, not for rendering UI.

→ React first renders the component Then runs the effect.

In which it is present

When you do any change in main function, Then in (Parent) re-rendering child function also re-renders.

So, if you change or increase counter Then in re-rendering, colorful also changes.

(i.e colorful is inside main())

So, if you put statement outside the useEffect, it will be run everytime whenever the count changes

But Now We are using useEffect so it only runs when color changes

(8)

So, wherever there is Dom manipulation Put it in useEffect.

Now come to 4th commit →
(Q103 done and useMemo done)

→ As you can see, whenever there is change in count,

<Colorful/> is also rendery.

So, for not rendering it use use useMemo hook.

```
export default React.memo(colorful);
```

Now colorful will not Run on render whenever there is change in main fn. it will only renders when change in color (dependency).

useMemo hook

React hook That memorizes (caches) The result of a calculation so React doesn't recomputes it on every Render.

Date 9

~~It~~ mainly used to improve Performance when a calculation is expensive.

like here we use React useMemo useMemo hook with <Colorful />

exception →

useMemo will Render ~~The~~ The function when any dynamic Prop is Provided

ex → <Colorful name = "Hunny" />

↓

here static Prop.

b/c it does not changes

so, no re-rendering of colorful happens when main renders.

but in

count

<Colorful name = {~~count~~} />

↓

Since count will change whenever There is change in main or counter APP, so, This is dynamic Prop

so, here re-rendering of colorful also happen whenever change in count.

Page No.

(10)

Use usememo when Parent element is re-rendering frequently, There is no need for That component for re-rendering.

Otherwise not use usememo, b/c it will increase overhead.